Working with MTD Devices

This article shows how kernel and application developers (in C) can make use of MTD in Linux.

MTD (Memory Technology Devices) are NAND/NOR-based flash memory chips used for storing non-volatile data like boot images and configurations. Readers are cautioned not to get confused with USB sticks, SD cards, etc, which are also called flash devices, but are not MTD devices. The latter are generally found on development boards, used to store boot loaders, an OS, etc.

Even though MTD devices are for data storage, they differ from hard disks and RAM in several aspects. The biggest difference is that while hard disk sectors are re-writable, MTD device sectors must be erased before re-writing—which is why they are more commonly called erase-blocks. Second, hard disk sectors can be re-written several times without wearing out the hardware, but MTD device sectors have a limited life and are not usable after about 10^3-10^5 erase operations. The worn out erase-blocks are called bad blocks and the software must take care not to use such blocks.

Like hard disks, MTD devices can be partitioned and can therefore act as independent devices. On a system with one or more MTD devices, device and partition information can be obtained from the */proc/mtd* file. A typical */proc/mtd* file is as follows:

```
cat /proc/mtd
dev:   size   erasesize name
mtd0: 000a0000 00020000 "misc"
mtd1: 00420000 00020000 "recovery"
mtd2: 002c0000 00020000 "boot"
mtd3: 0fa00000 00020000 "system"
mtd4: 02800000 00020000 "cache"
mtd5: 0af20000 00020000 "userdata"
```

A partitioned MTD device can be depicted as in Figure 1, which shows the relation between an MTD device, a partition and a sector. As already said, MTD write operations are different from usual storage devices. Therefore, before we move further, let's understand how write operations take place on MTD devices. Figure 2 shows a typical write case. The left-most part shows a sector that has some data at the end. The rest of the sector has not been written since the last erase. A user wants to write 'new data 1' to this sector at offset 0. Since this part of the sector has already been erased, it is ready to be written and so 'new data 1' can be directly written to the sector. Later, the user may want to write 'new data 2', again at offset 0. To do this, the sector must be erased. Since the sector needs to be erased in entirety, the 'old data' must be backed up in a temporary buffer. After erasing the complete sector, the 'new data 2' and 'old data' must be written at appropriate offsets.

This procedure is the reason there are specific file systems for MTD devices, like JFFS2 and YAFFFS, and flash translation layers (FTL) like NFTL, INFTL, etc. These FTLs and file systems take special care of MTD device properties to hide complexity from the user.

In the first section that follows, we will look at how to access, read/write and erase MTD devices from Linux applications. The second section describes the same things in kernel space, so that this article can be useful to both application as well as kernel developers.

Accessing MTDs from applications

The user must know the device partition to work upon, which can be found from */proc/mtd* as shown earlier. Assuming users want to work on the *"userdata"* partition, they must use the */dev/mtd5* device.

The first thing to do is to get information about the MTD device. Use the *MEMGETINFO ioctl* command, as follows:

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <mtd/mtd-user.h>

int main()
{
    mtd_info_t mtd_info;
    int fd = open("/dev/mtd5", O_RDWR);
```